S02.E1 - Ejercicio 01 | Sesión 02

Window Functions y Programación Procedural en PostgreSQL

Entorno de Trabajo: Base de datos *demodb* (Esquema *bookings*)
Duración Estimada: 25 Minutos (5 min teoría + 20 min desarrollo práctico)
Modalidad: Individual o en Parejas
Objetivo: Dominar la analítica con Window Functions (particionado, orden e interpolación de filas) y la creación de funciones procedurales en PL/pgSQL para la encapsulación de lógica de negocio.

Parte 1: Análisis Teórico (5 minutos)

Responde de forma concisa a las siguientes dos cuestiones de arquitectura de base de datos:

Cuestión 1.1 (Window Functions vs. GROUP BY): Explica la diferencia técnica fundamental entre realizar un cálculo de agregación utilizando la cláusula GROUP BY y realizarlo utilizando una Window Function con OVER (PARTITION BY ...). ¿Por qué las Window Functions conservan la granularidad original de las filas? Indica en qué fase del orden de ejecución lógica de PostgreSQL actúa la cláusula OVER().

Cuestión 1.2 (Control Transaccional en PL/pgSQL): ¿Por qué un Procedimiento Almacenado (CREATE PROCEDURE) permite ejecutar instrucciones explícitas de control transaccional en caliente (COMMIT; y ROLLBACK;) dentro de su cuerpo, mientras que una Función definida por el usuario (CREATE FUNCTION) lo prohíbe terminantemente?

Parte 2: Caso Práctico en demodb (20 minutos)

Escenario Operativo: El departamento de inteligencia de negocio de la aerolínea desea evaluar el comportamiento tarifario de los billetes por vuelo y construir una función procedural reutilizable que extraiga el perfil analítico de los pasajeros.

Tarea 1: Ranking Tarifario y Diferencial con Window Functions (10 min)

Escribe una consulta SQL sobre la tabla bookings.ticket_flights (tf) unida a la tabla bookings.flights (fl) que devuelva las siguientes columnas:
• fl.flight_id y fl.flight_no
• tf.fare_conditions y tf.amount (precio del billete)
• **posicion_precio:** Posición ordinal del billete dentro de su mismo vuelo ordenado de mayor a menor precio, usando ROW_NUMBER() OVER (...).
• **diferencia_con_anterior:** Diferencia entre el precio del billete anterior más costoso y el precio del billete actual dentro del mismo vuelo, utilizando la función de navegación LAG().
• **promedio_vuelo:** Precio promedio de los billetes de ese vuelo específico mediante AVG(tf.amount) OVER (...).

Restricciones: Considera únicamente los vuelos con estado status = 'Arrived'. Limita la salida final a los primeros 100 resultados ordenados por flight_id y posicion_precio.

Tarea 2: Función Procedural PL/pgSQL para Resumen de Pasajero (10 min)

Diseña y crea una función en lenguaje PL/pgSQL llamada bookings.get_passenger_stats(p_passenger_id varchar) que devuelva una tabla (RETURNS TABLE) con la siguiente estructura de salida:
• passenger_id (varchar)
• total_vuelos (integer)
• monto_total_gastado (numeric)
• clase_preferida (varchar)

Requisitos de Lógica Interna:
1. Calcular el total de vuelos en los que ha embarcado o reservado el pasajero.
2. Calcular la suma total del importe pagado en todos sus billetes (SUM(tf.amount)).
3. Determinar la condición de tarifa (fare_conditions) más utilizada por el pasajero (mediante subconsulta con GROUP BY fare_conditions ORDER BY COUNT(*) DESC LIMIT 1).
4. Comprobar la función invocándola con el pasajero de prueba '1234 567890' o similar.